

Project Title
Dictionary Query Engine with Maximum Edit Distance (Fuzzy Search)

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by

Bhanu Jakka	2520030006
Akshita Anand	2520030043
Harini P	2520030177

Team Number: 11
Section: 7

Under the guidance of

Dr S Madhavi Professor,
CSE Department



Hyderabad-500090, Telangana, India.

ABSTRACT

Spelling mistakes are common when users type sentences, and traditional exact matching cannot identify or correct misspelled words. This project presents a Smart Spell Checker that combines multiple string matching and edit distance algorithms to detect dictionary words, search for patterns, and provide spelling suggestions. The system uses a Trie to store dictionary words character by character, allowing efficient organization of words based on their common prefixes. Aho-Corasick is used to perform exact matching of multiple dictionary words in the input sentence, while the KMP algorithm is used for efficient single-pattern searching with the help of the Longest Prefix Suffix (LPS) array.

For spelling correction, the system uses Wagner-Fischer Edit Distance to calculate the minimum number of insertions, deletions, and substitutions required to transform a misspelled word into a suggested dictionary word. Damerau-Levenshtein Edit Distance is also used to handle adjacent character transpositions, which are common typing errors. The system compares misspelled words with the dictionary, identifies a suitable correction, and displays the calculated edit distances. By combining Trie, Aho-Corasick, KMP, and dynamic programming-based edit distance techniques, the project demonstrates how different data structures and string algorithms can work together to create an efficient and practical spelling correction system.

DESCRIPTION

Problem

Spelling mistakes can cause problems in text searching because exact matching fails when even a small number of characters are typed incorrectly. A user may enter words with missing characters, extra characters, replaced characters, or swapped adjacent characters. Therefore, a system is required that can perform exact searching as well as identify and suggest corrections for misspelled words.

Approach

The proposed Smart Spell Checker combines a Trie, multiple string matching algorithms, and edit distance algorithms to analyze the user's input sentence. A dictionary of words is stored in a Trie, where each node represents a character and words sharing common prefixes use the same path in the structure.

1. Trie

The Trie is used to store the dictionary words character by character. Each character is represented by a node, and a flag indicates whether a complete dictionary word ends at that node. This provides an organized structure for storing and accessing dictionary words and also forms the base structure used by the Aho-Corasick algorithm.

2. Aho-Corasick Exact Matching

The Aho-Corasick algorithm is used to search for multiple dictionary words in the input sentence. It uses failure links to efficiently continue matching when a mismatch occurs instead of starting the search again from the beginning. The algorithm reports the dictionary words that are exactly present in the input sentence.

3. KMP Pattern Searching

The Knuth-Morris-Pratt (KMP) algorithm is used for single-pattern searching. The user enters a pattern, and the system searches for that pattern in the input sentence. KMP uses the Longest Prefix Suffix (LPS) array to avoid unnecessary character comparisons after a mismatch, making the search more efficient.

4. Wagner-Fischer Edit Distance

The Wagner-Fischer algorithm is used to calculate the standard Levenshtein edit distance between a misspelled word and a suggested dictionary word. It uses dynamic programming to find the minimum number of insertions, deletions, and substitutions required to transform one word into another.

5. Damerau-Levenshtein Edit Distance

The Damerau-Levenshtein algorithm extends the edit distance calculation by also considering adjacent character transpositions. This is useful for common typing mistakes where two neighboring characters are accidentally swapped. The system calculates this distance along with the Wagner-Fischer distance when generating spelling suggestions.

Spell Correction

After identifying the words in the sentence, the system checks each word against the dictionary. If a word is not found, the system compares it with dictionary words using Damerau-Levenshtein distance and selects a suitable closest word as a spelling suggestion. The corresponding Wagner-Fischer and Damerau-Levenshtein distances are then displayed.

Outcome

The system provides a complete demonstration of exact matching, pattern searching, and spelling correction using five important algorithms. It identifies dictionary words using Aho-Corasick, searches for a user-specified pattern using KMP, and detects spelling mistakes using dynamic programming-based edit distance techniques. The project demonstrates how multiple data structures and algorithms can be combined to build a practical Smart Spell Checker.

```
=====
=
                        SMART SPELL CHECKER
=====
=
Enter a sentence to check:
```

```
Enter a sentence to check: helo wrld teh javaa programing
algorithm
```

```
=====
=
                STEP 1 - AHO-CORASICK EXACT MATCHING
=====
=
[NOT FOUND] No dictionary words found.
```

```
=====
=
                STEP 2 - KMP PATTERN SEARCH
=====
=
Enter a pattern to search using KMP: java
[OK] Pattern found at 0-based position: 14
LPS Array: [0, 0, 0, 0]
```

```
=====
=
                STEP 3 - SPELL CORRECTION
                WAGNER-FISCHER + DAMERAU-LEVENSHTEIN
=====
=

[MISSPELLED] helo
  Suggestion           : hello
  Wagner-Fischer distance : 1
  Damerau distance      : 1

[MISSPELLED] wrld
  Suggestion           : world
  Wagner-Fischer distance : 1
  Damerau distance      : 1

[MISSPELLED] teh
  Suggestion           : the
  Wagner-Fischer distance : 2
  Damerau distance      : 1
```

[MISSPELLED] javaa

Suggestion : java

Wagner-Fischer distance : 1

Damerau distance : 1

[MISSPELLED] programing

Suggestion : programming

Wagner-Fischer distance : 1

Damerau distance : 1

[MISSPELLED] algoritm

Suggestion : algorithm

Wagner-Fischer distance : 1

Damerau distance : 1

=====

=

ALL ALGORITHMS COMPLETED

=====

[1] Run the program again

[2] Exit

Enter your choice: 1

[OK] Starting a new program run...

=====

=

SMART SPELL CHECKER

=====

=

Enter a sentence to check: